Image Fragment Reconstruction via Adjacency Prediction

May 3, 2026

Contents

1 Problem Statement

Each batch contains $N = 10$ source images of resolution $64 \times 64 \times 3$. Every image is cut into a regular 4×4 grid of non-overlapping 16×16 fragments, and the resulting $M = 160$ fragments from all 10 images are pooled into a single unordered collection. The model must partition the collection into 10 groups such that all fragments in a group come from the same source image.

Two constraints make the task non-trivial. First, the original grid coordinates of each fragment are withheld at inference, so absolute position cannot be used as a feature. Second, the source-image label of each fragment, which we denote $s(k) \in \{1, \ldots, 10\}$, is withheld during training, so the model has to learn from structural properties of the fragments rather than from direct supervision. Two facts about the partition are known and may be exploited: there are exactly 10 clusters, and each contains exactly 16 fragments.

Crucially, although the source labels are hidden, *spatial adjacency* within a source image is fully determined by the grid: two fragments at grid positions (r, c) and (r', c') within the same source image are adjacent iff $|r - r'| + |c - c'| = 1$. This adjacency relation is therefore a free-of-charge supervisory signal — and the one our pretext task exploits (Section 2).

2 Related Work

The problem sits at the intersection of self-supervised representation learning and clustering. We review three families of prior work that informed the approach.

Pretext-task self-supervision. The dominant paradigm in self-supervised vision until the rise of contrastive methods was to define a hand-crafted pretext task whose labels are derivable from the data itself. Noroozi and Favaro (2016) solve a jigsaw permutation: given a 3×3 grid of fragments shuffled by one of 100 fixed permutations, predict which permutation was used. The encoder learns features useful for downstream classification. Our adjacency prediction is a much simpler, binary variant of this idea — rather than predicting a discrete permutation index over an exponentially large space, we predict a binary label per pair. This trades off representational richness for sample efficiency.

Contrastive learning. SimCLR (Chen et al. 2020) and MoCo (He et al. 2020) learn representations by pulling together augmented views of the same image and pushing apart views of different images. In the present setting these methods are awkward: there is no natural positive pair construction, since the natural "view" of a fragment is the fragment itself. One could treat all fragments from one source image as positives of each other, but this requires the source label that we are explicitly forbidden to use during training. Adjacency labels, in contrast, are derivable from the grid structure alone.

Clustering-based self-supervision. DeepCluster (Caron, Bojanowski, et al. 2018) alternates between clustering current embeddings and using cluster assignments as pseudo-labels. SwAV (Caron, Misra, et al. 2020) extends this with online clustering and equipartitioned cluster assignments via the Sinkhorn algorithm. Both are conceptually well-suited to this task, but they introduce significant complexity (alternating training, prototype maintenance) for what is at heart a small-scale clustering problem with $N = 10$ images. We treat this family as a strong candidate for follow-up work but defer it.

3 Core Idea

The central observation is that two fragments cut from the same image and placed next to each other in the grid share a common boundary — colours bleed over, textures continue, and objects span tiles. This visual continuity is a strong and reliable signal that does not require any human annotation to exploit: since the experimenter creates the grid, the ground-truth adjacency of every fragment pair is known at all times.

The proposed approach uses adjacency prediction as a **pretext task** (Noroozi and Favaro 2016). The model is trained to solve a surrogate problem — predicting whether two fragments are spatially adjacent — and in doing so is forced to learn visual representations that capture source identity as a side effect. At inference time the pretext objective is discarded and the learned representations are used for clustering.

This is self-supervised throughout: no labels beyond those derivable from the grid structure are ever used.

4 Architecture

The model is a Siamese network: a shared encoder is applied to each of two input fragments, and a comparison head maps the two embeddings to a confidence score $p_{ij} \in [0, 1]$ for the binary question of interest (Figure 1). When the multi-task loss is enabled, a second comparison head with the same structure is added for the same-image task.

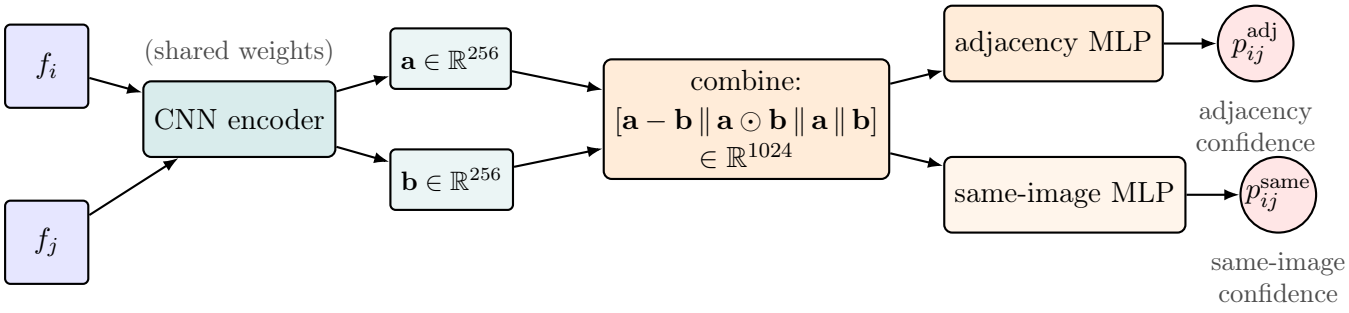

Figure 1: Schematic of the model. The CNN encoder is applied with shared weights to each of two input fragments; its 256-dimensional embeddings are combined into a single 1024-dimensional vector via concatenation of difference, element-wise product, and the two embeddings themselves; this vector is fed to one or two comparison heads (adjacency, same-image) that produce a confidence score $p_{ij} \in [0, 1]$ for the corresponding binary question.

The components are:

Shared CNN encoder. A single convolutional neural network maps each 16×16×3 fragment to a 256-dimensional embedding vector. The architecture is three convolutional blocks (Conv–BatchNorm–ReLU), with max-pooling after blocks two and three, reducing the spatial dimensions from 16×16 to 4×4. The flattened $4 \times 4 \times 128$ feature map is projected to a 256-dimensional embedding via a linear layer followed by ReLU and dropout ($p = 0.3$).

The encoder is **shared** — the same weights are used for both fragments in a pair. This is the defining property of a Siamese network (Bromley et al. 1994): weight sharing ensures that the comparison is symmetric and that the encoder learns a universal fragment representation rather than one tuned to a specific input slot.

Comparison head (MLP). Let $\mathbf{a} = \mathrm{CNN}(f_i)$ and $\mathbf{b} = \mathrm{CNN}(f_j)$ denote the two embeddings produced by the shared encoder for an input pair. The comparison head takes as input the 1024-dimensional vector

$$\mathbf{z}_{ij} = \big[\mathbf{a} - \mathbf{b} \parallel \mathbf{a} \odot \mathbf{b} \parallel \mathbf{a} \parallel \mathbf{b}\big] \in \mathbb{R}^{1024},$$

formed by concatenating ($\parallel$) four sub-vectors of size 256: the element-wise difference $\mathbf{a} - \mathbf{b}$, capturing direction and magnitude of dissimilarity; the element-wise (Hadamard) product $\mathbf{a} \odot \mathbf{b}$, capturing feature co-activation (large only when both fragments score high on the same feature); and the two raw embeddings, preserving each fragment's individual information. This four-term combination is borrowed from the InferSent sentence-pair classification model of Conneau et al. (2017), where it was shown to outperform plain concatenation on natural-language inference. We adopt it here without ablation in our setting; whether the explicit difference and product terms help on image fragments, or whether a simpler $[\mathbf{a}, \mathbf{b}]$ concatenation with a larger hidden layer would do as well, is left as future work. The MLP maps $\mathbf{z}_{ij}$ through that hidden layer (ReLU) to a single scalar in $[0, 1]$:

$$p_{ij} = \sigma\big(W_2 \,\mathrm{ReLU}(W_1 \,\mathbf{z}_{ij} + b_1) + b_2\big).$$

5 Training

5.1 Constructing Training Labels

From a sample of 10 images cut into a 4×4 grid, each fragment has at most 4 adjacent neighbours (fewer at corners and edges). For a sample of 160 fragments the total number of pairs is:

$$\binom{160}{2} = 12{,}720$$

Each pair receives a binary label: $y_{ij} = 1$ if the two fragments are adjacent within the same image, $y_{ij} = 0$ otherwise. This label is derived entirely from the known grid positions and requires no human annotation (Figure 2).

5.2 Loss Function

Binary cross-entropy is the natural loss for a binary prediction task:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{N} \sum_{(i,j)} \Big[y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big]$$

Positive (adjacent) pairs are far rarer than negative pairs in this configuration. With 10 source images on a 4×4 grid, each image contributes $4 \cdot 3 + 3 \cdot 4 = 24$ adjacent pairs, giving $n_+ = 240$ positives in total. The remaining pairs are negatives: $n_- = \binom{160}{2} - 240 = 12{,}480$. The ratio $n_-/n_+ = 52$ is therefore a constant determined by the grid geometry, not a

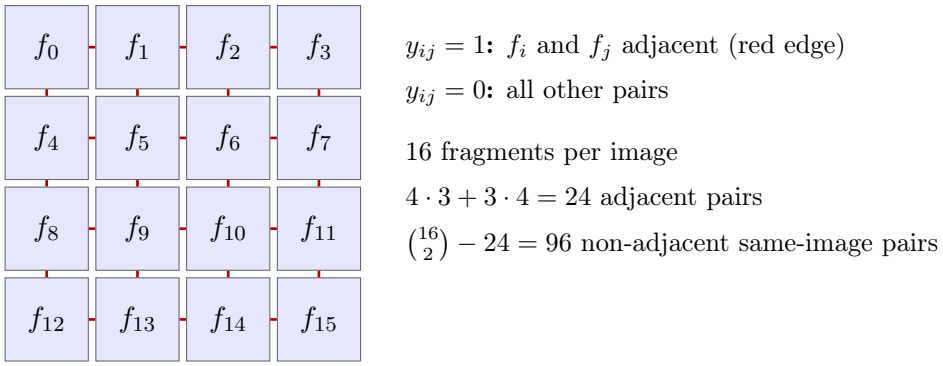

Figure 2: Adjacency labelling within a single source image. Each of the 16 fragments is connected by a red edge to each of its grid neighbours. The 24 highlighted edges are the positive pairs ($y_{ij} = 1$); all other pairs within the image (96 of them) and across images are negative ($y_{ij} = 0$).

per-batch quantity. Vanilla BCE on this data is dominated by the negative class: the gradient is overwhelmingly driven by the loss on non-adjacent pairs, and the model has little incentive to identify the few adjacent ones. The standard remedy is to upweight the rare class. Since only the ratio of the two class weights matters, we parameterise the loss with a single *tilt* parameter $\beta \geq 1$:

$$\mathcal{L}_{\text{WBCE}}(\beta) = -\frac{1}{N} \sum_{(i,j)} \Big[\beta \cdot \tfrac{n_-}{n_+} \, y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big].$$

The textbook value is $\beta = 1$, the class-balanced loss: it equalises the total gradient contribution of the two classes, treating a single positive as worth 52 negatives. Setting $\beta > 1$ over-weights positives beyond this point — a positive mistake is now *more* costly than the natural class ratio implies. For our downstream task this is desirable, and we expect the optimum to lie at a moderate-to-aggressive tilt.

Why we lean toward a moderate-to-aggressive tilt. The downstream task is graph clustering, not per-edge classification. False negatives directly weaken intra-cluster connectivity, while false positives are only harmful when they connect fragments from *different* source images — a case the model rarely produces because cross-image fragments are visually dissimilar. Same-image false positives, the more common kind, actually reinforce the connectivity spectral clustering relies on. The error costs are therefore asymmetric in favour of recall, and we expect the optimal β to lie above the class-balanced point.

5.3 Computational Cost

The CNN encoder runs once per fragment (160 forward passes per sample). All pairwise embeddings are then combined using tensor operations, and the lightweight MLP processes all 12,720 pairs. Since the expensive step (the CNN) runs only 160 times, the computational overhead is manageable.

6 From Pretext Task to Clustering

After training, the model produces a value $p_{ij} \in [0, 1]$ for every pair of fragments. We do not interpret this as a calibrated probability — the model is trained on a binary cross-entropy objective but its outputs are not guaranteed to be calibrated to true frequencies. We use p_{ij} instead as a confidence score: the higher the value, the more confident the model is that fragments i and j are adjacent. These scores define a **fully connected weighted graph** over the 160 fragments, where edge weight $w_{ij} = p_{ij}$ reflects the model's confidence that the two fragments are adjacent — and by extension, that they originate from the same image (Figure 3).

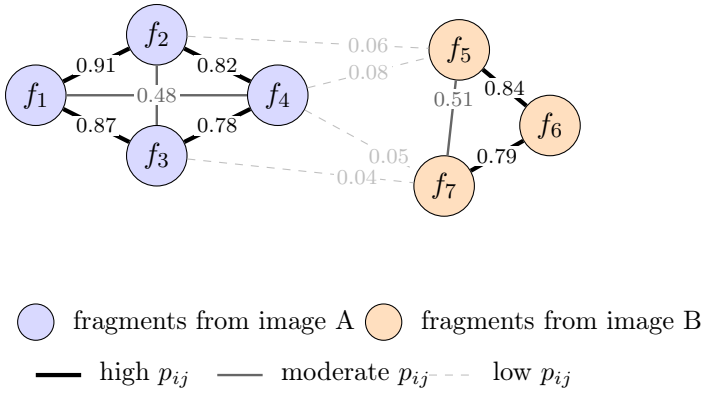

Figure 3: Schematic of the predicted similarity graph. Each node is a fragment; edge weight p_{ij} is the model's confidence that the two fragments are adjacent. Edges within a true source image (here, two illustrative source images shown in blue and orange) tend to receive high weight; cross-image edges receive low weight. Spectral clustering recovers the source-image partition by finding the cut that separates the densely-connected sub-graphs.

Balanced spectral clustering. **Spectral clustering** is the natural choice for a weighted graph. Given the 160×160 similarity matrix W with $w_{ij} = p_{ij}$, define the diagonal degree matrix D with $D_{ii} = \sum_j w_{ij}$ and the symmetric normalised graph Laplacian

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

The spectral embedding stacks the k leading eigenvectors of L_{sym} into a $160 \times k$ matrix; in the eigenspace, fragments belonging to densely-connected sub-graphs lie close together. The standard pipeline then runs k-means on this embedding to obtain hard cluster assignments.

Plain k-means ignores the fact that each true cluster contains exactly $G^2 = 16$ fragments and can produce groups of very unequal size. We enforce balance by replacing the k-means step with a balanced assignment: build a $160 \times (k \cdot G^2)$ cost matrix in which each cluster appears G^2 times, and solve the resulting linear assignment problem with the Hungarian algorithm[1]. Cluster centroids are then re-estimated and the assignment iterated until convergence (typically a handful of iterations). The result is a partition of the 160 fragments into 10 clusters of size 16, by construction.

[1]Implemented using `sklearn.manifold.SpectralEmbedding` for the eigendecomposition and `scipy.optimize.linear_sum_assignment` for the assignment.

7 Evaluation Metric for the Clustering Task

The pretext task (adjacency prediction) and the downstream task (clustering of fragments by source image) are evaluated with different metrics. Adjacency prediction is evaluated with the threshold-free metrics AUROC and AUPRC (described below in the multi-batch averaging paragraph); the downstream clustering is evaluated with the **Adjusted Rand Index (ARI)**. We focus on the clustering metric here since it is the headline number for this work.

Definition (Adjusted Rand Index). Let $\mathcal{C} = \{C_1, \ldots, C_N\}$ be the true partition of the fragments into $N = 10$ source-image groups, and $\hat{\mathcal{C}} = \{\hat{C}_1, \ldots, \hat{C}_N\}$ the partition predicted by the model. For each pair of fragments, look at whether the two are in the same true group and whether they are in the same predicted cluster:

- a = number of pairs in the same true group *and* the same predicted cluster
- b = number of pairs in different true groups *and* different predicted clusters
- c = number of pairs in the same true group but different predicted clusters
- d = number of pairs in different true groups but the same predicted cluster

The (unadjusted) Rand Index is the agreement rate $\mathrm{RI} = (a+b)/(a+b+c+d)$. The Adjusted Rand Index normalises this against the expected RI under random labellings of fixed cluster sizes:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}.$$

A random clustering scores $\mathrm{ARI} = 0$, a perfect one scores $\mathrm{ARI} = 1$, and a clustering worse than random can score below zero.

Why ARI rather than purity. ARI is preferred over simpler metrics like purity because purity only looks at the dominant true class in each predicted cluster and ignores how the remaining errors are distributed.

> **Example.** Suppose two predicted clusters each have 60% fragments from image A. The first additionally has 40% from image B; the second has 20% from B and 20% from C. Both clusters score the same purity (0.60), even though the second is visibly more disordered. ARI distinguishes the two cases because it counts pair-level agreements rather than looking only at the plurality label.

NMI (Normalised Mutual Information) is reported alongside ARI as a complementary metric. It measures how much information the predicted clusters share with the true groupings, normalised to $[0, 1]$.

Multi-batch averaging. Each evaluation step samples a fresh batch of 10 source images from the validation generator. Because individual batches vary substantially in difficulty (some draws of 10 ImageNet classes are visually well-separated, others contain near-duplicates), a single-batch ARI estimate has high variance. We average each metric over $n_{\mathrm{eval}} = 20$ fresh validation batches per evaluation step. This stabilises the training curves and, crucially, the early-stopping decision and the checkpoint-selection criterion, which would otherwise be biased toward lucky batches.

8 Experimental Setup

Data. ImageNet-64 (Chrabaszcz, Loshchilov, and Hutter 2017): a downsampled version of the ImageNet-1k classification dataset in which every image has been resized to 64×64 pixels. The dataset is split into a training partition and a test partition; we draw the model's training images from the former and the validation images from the latter. Each training step samples 10 random images on the fly and produces a fresh batch of 160 fragments; the data generator is therefore effectively infinite. The augmentation pipeline (rotation up to $\pm 20°$, horizontal/vertical shift up to 10%, channel shift 0.2) is applied to the full image *before* fragmentation and is provided by the dataset's data generator (`data.py`); it is not modified.

Hardware. All training and evaluation reported in this document was performed on a MacBook Air (Apple M-series CPU). PyTorch is configured to fall back to CPU; no GPU is required. A full training run completes in approximately 20–25 minutes on this hardware.

Reproducibility.

- Each run writes its full configuration, training log, and best checkpoint into `runs/{run_name}/`, and appends a one-line summary to `results.jsonl`. The configuration is therefore self-contained and re-runnable.
- Random seeds are fixed for the clustering routines[2]. The PyTorch random seed is *not* currently fixed, so model initialisation and the order of training batches vary between runs; this is by design — repeated runs with different seeds give a sense of the variance in final ARI.
- Hyperparameters and run metadata are versioned in `config.yaml`.

Hyperparameters. The complete list of hyperparameters is maintained in `config.yaml` and may still change in the course of further ablations. The current values are documented there; this section will be replaced with a final hyperparameter table once the study is complete.

9 Results

9.1 Headline number

The best configuration we found uses a multi-task loss with equal weights on the adjacency and same-image heads, plain BCE on both, balanced spectral clustering, and validation ARI averaged over 20 fresh batches. It achieves

$$\text{best ARI} = 0.570, \qquad \text{adjacency AUROC} = 0.959, \qquad \text{adjacency AUPRC} = 0.328$$

on the validation generator. We estimate the run-to-run noise floor (variation between repeats of the same configuration with different random seeds) at approximately ± 0.04 ARI, so a difference of less than ~ 0.04 between two runs should be treated as inconclusive.

[2] `random_state = 42` in `sklearn.cluster.SpectralClustering`, `sklearn.cluster.KMeans`, and `sklearn.manifold.SpectralEmbedding`.

9.2 Effect of the loss tilt parameter β

The first ablation sweeps the WBCE tilt over $\beta \in \{0.019, 0.3, 1.0\}$: plain unweighted BCE ($\beta \cdot 52 = 1$), a mild upweighting, and the textbook class-balanced weighting ($w_+ = 52$).

Run	β	Best ARI	AUROC / AUPRC
`plain_bce`	0.019	0.535	0.969 / 0.432
`wbce_beta_03`	0.3	0.523	0.973 / 0.435
`wbce_beta_1`	1.0	0.462	0.969 / 0.405

AUROC barely moves across two orders of magnitude in β: the model's *ranking* of pairs is essentially independent of the loss weighting. What changes is calibration, which spectral clustering picks up indirectly through the magnitude of the edge weights. ARI varies more (by ~ 0.07 between best and worst), but a substantial part of that is run-to-run noise. The cleanest reading is that β is a second-order knob across the range we tested, with plain BCE performing at least as well as any weighted variant. We adopt $\beta = 1/52$ (plain BCE) for all subsequent experiments.

This contradicts the prior expectation we expressed in Section 6 that an aggressive tilt $\beta \in [1.5, 3]$ would help. The asymmetric-error-cost argument is not wrong in principle, but the model already learns a near-optimal ranking under plain BCE (AUROC ≈ 0.97), so additional reweighting has little remaining effect on the downstream clustering.

9.3 Effect of the multi-task weight λ_{same}

The second sweep adds a same-image prediction head to the model, weighted by λ_{same} relative to the adjacency loss ($\lambda_{\text{adj}} = 1$ throughout). The same-image label $y_{ij}^{\text{same}} = \mathbb{1}[s(i) = s(j)]$ is also free, derivable from the labels passed to the data generator, and denser ($\sim$10% positives instead of 1.9%). At inference, when the same-image head is active we use *its* confidence scores as the affinity matrix for spectral clustering, since they are more directly aligned with the downstream objective.

Run	λ_{same}	Best ARI	AUROC / AUPRC
`plain_bce_baseline`	0.0	0.554	0.970 / 0.443
`multitask_01`	0.1	0.489	0.954 / 0.308
`multitask_05`	0.5	0.549	0.959 / 0.331
`multitask_10`	1.0	**0.570**	0.959 / 0.328
`multitask_15`	1.5	0.566	0.956 / 0.314

The trend is non-monotonic. A small auxiliary same-image weight ($\lambda = 0.1$) actually *hurts* ARI compared to adjacency-only (0.489 vs. 0.554): the same-image task is dense enough that even at this weight it occupies a non-trivial share of the gradient direction, while contributing too little to the encoder for that share to be useful. Around $\lambda \in [1.0, 1.5]$ the two losses are roughly balanced and ARI peaks at 0.570 — a +0.016 gain over adjacency-only, at the edge of our ± 0.04 noise floor but pointing in the direction we expected.

The adjacency AUROC drops slightly (from 0.970 to 0.959) and the AUPRC drops more (from 0.443 to 0.328) when same-image is added, because half the gradient budget now goes to a different objective. The multi-task model's adjacency predictions get worse

on the per-edge metrics, but the clustering metric improves — consistent with the model trading per-edge precision for the denser same-image signal that spectral clustering can use directly.

9.4 Ablation: same-image only

To isolate the contribution of the adjacency task we ran one further experiment with $\lambda_{\text{adj}} = 0$, i.e., training only on same-image prediction.

Run	$(\lambda_{\text{adj}}, \lambda_{\text{same}})$	Best ARI	adjacency AUROC
`plain_bce_baseline`	$(1, 0)$	0.554	0.970
`multitask_10`	$(1, 1)$	**0.570**	0.959
`same_only`	$(0, 1)$	0.502	0.500

With $\lambda_{\text{adj}} = 0$ the adjacency head receives no gradient at all, which is reflected in its AUROC sitting at chance (0.500) and AUPRC at the positive rate (0.019, not shown). On the downstream task the model still clusters meaningfully (ARI 0.502, far above random), so the encoder does *not* collapse to a within-image-degenerate solution as we feared was a risk. But it does drop 0.07 ARI compared to the multi-task model and 0.05 compared to adjacency-only, both of which are larger than the noise floor.

The takeaway: the adjacency pretext is doing real, measurable representation work that same-image alone cannot replicate. The two signals are *complementary* — adjacency provides a fine-grained relational signal that prevents the encoder from collapsing within an image; same-image provides the dense task-aligned signal that spectral clustering benefits from directly. The combination wins.

9.5 Apparent regression from earlier numbers

An earlier full training run (before multi-batch eval and balanced clustering were introduced) reported a best ARI of 0.787. The current best of 0.570 is lower not because the model regressed, but because the earlier evaluation used a single validation batch per step. Within-batch variance was large ($\sim \pm 0.1$), and the 0.787 value was a single lucky draw. With averaging over twenty fresh batches per evaluation, the headline number reflects the model's actual generalisation behaviour rather than a favourable sample.

10 Failure Modes

Figure 4 shows four misclassified fragments from the best checkpoint. The model produces only an unordered partition into $k = 10$ clusters with arbitrary integer IDs; to relate them to source images for this visualisation we align cluster IDs to true source IDs by Hungarian matching on the validation batch (`scipy.optimize.linear_sum_assignment` on the 10×10 confusion matrix). The rightmost column then shows the source image whose fragments form the plurality of the cluster the misclassified fragment was assigned to. The matching is per-batch and uses the true labels; the model itself has no notion of source identity.

A reliable taxonomy of failure modes would require aggregating misclassified fragments across many batches, which we leave as follow-up work.

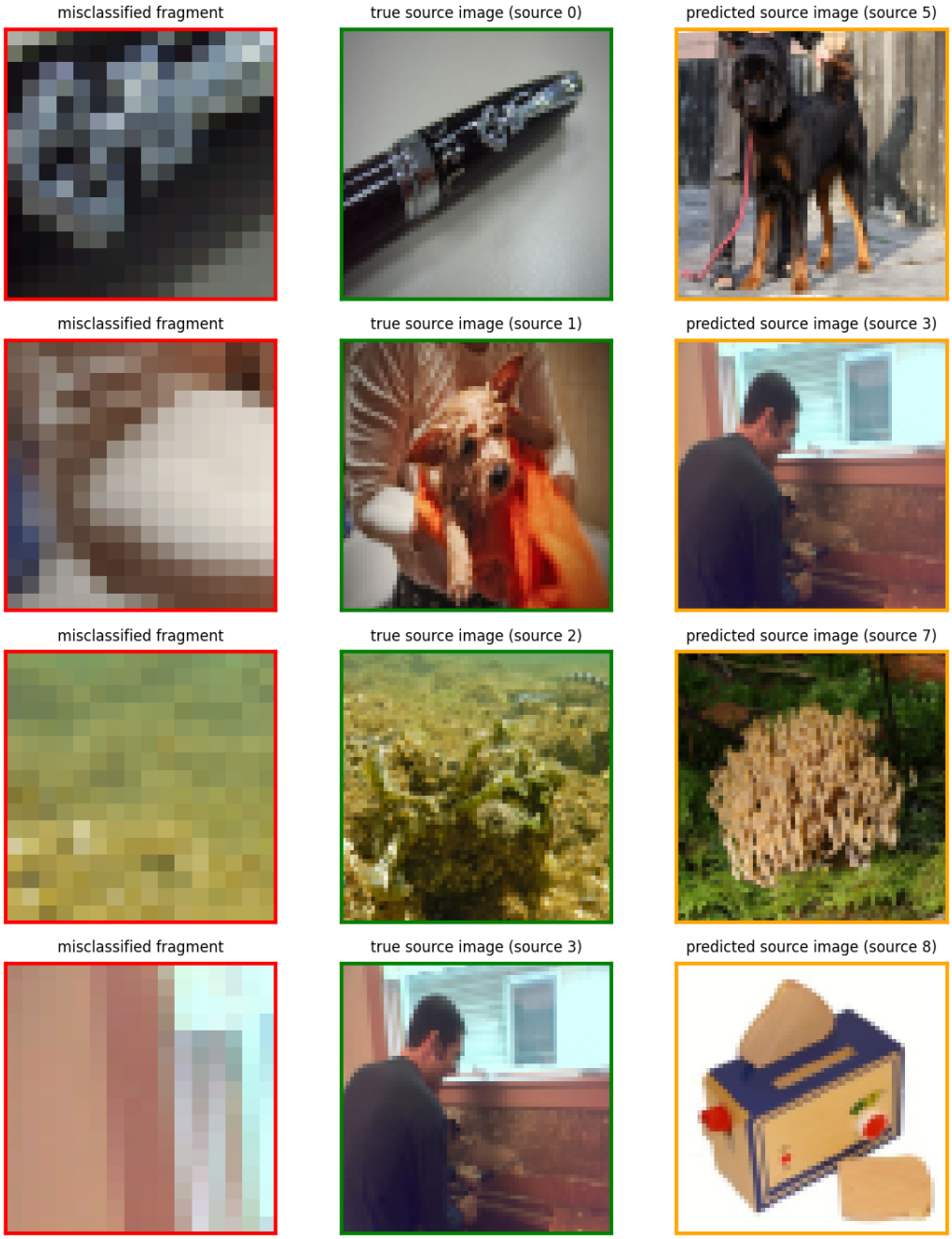

Figure 4: Selected misclassified fragments from the best checkpoint (`multitask_10`, best ARI $= 0.570$, $\beta = 0.01923$, $\lambda_{\text{adj}} = 1.0$, $\lambda_{\text{same}} = 1.0$). On the validation batch shown, 51 of 160 fragments were assigned to the wrong cluster after Hungarian matching of predicted to true labels. Each row shows: the misclassified fragment (left, red), the true source image (middle, green), and the source image the model assigned it to (right, orange).

11 Decision Points Summary

Decision	Choice	Alternatives considered
Training objective	Multi-task: adjacency + same-image	Adjacency only; same-image only;
Architecture	Siamese CNN + two MLP heads	Single encoder; pretrained encoder
Embedding combination	$[\mathbf{a}-\mathbf{b} \,\|\, \mathbf{a}\odot\mathbf{b} \,\|\, \mathbf{a} \,\|\, \mathbf{b}]$ (InferSent-style)	$[\mathbf{a}, \mathbf{b}]$ only (not ablated)
Loss function	Plain BCE on both heads	Weighted BCE (ablated, no gain);
λ_{same} weight	1.0 (equal to adjacency)	$\{0.1, 0.5, 1.5\}$ ablated; 1.0 wins
Clustering algorithm	Balanced spectral clustering	Plain spectral; k-means
Number of clusters	$k = 10$ (known)	Discovered automatically
Cluster size	Enforced ($G^2 = 16$) via Hungarian	Free
Eval batches per step	20 (averaged)	1 (single batch)
Adjacency metrics	AUROC, AUPRC (threshold-free)	F1 / precision / recall at fixed thr
Clustering metrics	ARI (primary), NMI	Purity

12 Limitations

- Adjacency is a local signal — it says nothing directly about fragments that are far apart within the same image. The model must generalise from local adjacency to global source identity.
- The graph has 12,720 edges. For larger grids or larger samples this grows quadratically and may become a bottleneck.
- The pretext metric and the task metric are misaligned. We train binary adjacency (only ∼1.9% positives) but evaluate clustering. The multi-task same-image head partially closes this gap, but at the cost of about 0.01 AUROC and 0.10 AUPRC on the adjacency task itself. The model trades per-edge precision for a denser task-aligned signal.

13 Further Suggestions

Differentiable surrogate for ARI. The training loss is on adjacency (or same-image), not on the downstream clustering objective. A more direct alternative is to optimise a differentiable approximation of ARI: the eigendecomposition and cluster-assignment steps of spectral clustering can be relaxed via Sinkhorn iterations (as in SwAV) or via a MinCut-style differentiable graph pooling. This would couple the representation learning to the clustering objective explicitly. We did not pursue it because the multi-task same-image head already captures most of the pairwise structure that ARI ultimately depends on, at a fraction of the implementation cost.

Self-supervised pretrained encoder. DINOv2 (He et al. 2020) or another self-supervised image encoder could replace our trained-from-scratch CNN. Since these encoders use no labels, this stays within the constraints of the task. Whether the gain (richer features) outweighs the loss (features were trained at 224×224, not 16×16) is an open question.

Topology-aware clustering. The true partition is more constrained than "equal sizes": each cluster's induced subgraph is isomorphic to a $G \times G$ grid (16 nodes, 24 edges, a planar graph with maximum degree 4). A clustering algorithm that exploited this structure, e.g.

a small GNN trained to find equipartitions whose induced subgraphs are grid-like, could disambiguate borderline assignments that pure spectral clustering misses. Implementation cost is significant; the marginal benefit beyond what the model's already-strong soft signal recovers is unclear.

Color-histogram baseline. A non-learned baseline using per-fragment color histograms with chi-square or EMD distance, followed by spectral clustering, would anchor the learned model's ARI against a meaningful reference. We expect this to be competitive on uniform-region fragments and weak on textured ones; a controlled comparison would quantify how much of the 0.57 ARI comes from learned representation vs. trivial color matching.

References

Bromley, Jane et al. (1994). "Signature Verification Using a Siamese Time Delay Neural Network". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 6, pp. 737–744.

Caron, Mathilde, Piotr Bojanowski, et al. (2018). "Deep Clustering for Unsupervised Learning of Visual Features". In: *European Conference on Computer Vision (ECCV)*, pp. 132–149.

Caron, Mathilde, Ishan Misra, et al. (2020). "Unsupervised Learning of Visual Features by Contrasting Cluster Assignments". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33, pp. 9912–9924.

Chen, Ting et al. (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *International Conference on Machine Learning (ICML)*, pp. 1597–1607.

Chrabaszcz, Patryk, Ilya Loshchilov, and Frank Hutter (2017). "A Downsampled Variant of ImageNet as an Alternative to the CIFAR Datasets". In: *arXiv preprint arXiv:1707.08819*.

Conneau, Alexis et al. (2017). "Supervised Learning of Universal Sentence Representations from Natural Language Inference Data". In: *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 670–680.

He, Kaiming et al. (2020). "Momentum Contrast for Unsupervised Visual Representation Learning". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9729–9738.

Noroozi, Mehdi and Paolo Favaro (2016). "Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles". In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 69–84.